

Arquitectura de Agentes LLM

Módulo 10 · Sistemas Agénticos y Multi-Agente

ReAct, plan-and-execute, reflexión, memoria persistente, HITL y LangGraph como framework de referencia.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Arquitectura de Agentes LLM

La arquitectura de un agente LLM determina cómo los componentes de percepción, razonamiento, memoria, y acción se conectan y coordinan. Hay múltiples patrones arquitectónicos posibles, cada uno con sus fortalezas y debilidades. La elección de la arquitectura correcta para cada caso de uso es una de las decisiones de diseño más importantes al construir sistemas agénticos. Anthropic, en su guía de construcción de agentes efectivos, establece un espectro desde "workflows" (flujos predefinidos donde el LLM completa pasos discretos) hasta "agentes" completamente autónomos (el LLM decide dinámicamente su flujo).

En 2025, los patrones arquitectónicos más establecidos para sistemas agénticos en producción son: el agente reactivo (ReAct loop básico), el agente con planificación explícita (plan-and-execute), el agente reflexivo (con auto-mejora), y el agente con memoria persistente. Para sistemas multi-agente, los patrones son: orquestador-trabajador (supervisor), pipeline de agentes, y debate/consenso entre agentes.

Los patrones arquitectónicos principales

Patrón 1: Agente reactivo (ReAct loop)

El patrón más simple: el agente recibe el objetivo y entra directamente en el loop Thought → Action → Observation sin planificación explícita previa. Decide en cada paso qué herramienta usar basándose en el estado actual. Es reactivo: responde a lo que observa en cada iteración sin un plan a largo plazo. Adecuado para tareas relativamente simples donde no se necesita planificación multi-paso o para exploración abierta donde el plan depende totalmente de los resultados intermedios.

Patrón 2: Plan-and-Execute

El agente primero genera un plan completo (lista de pasos) y luego ejecuta cada paso con un agente executor más simple. El agente planificador tiene una visión global del objetivo; el agente executor está especializado en la ejecución eficiente de acciones concretas. La ventaja es mejor coherencia global (el plan mantiene el hilo conductor) y facilidad de debugging (el plan es visible). La desventaja es menor adaptabilidad: si los pasos iniciales del plan producen resultados inesperados, el plan puede volverse obsoleto.

Patrón 3: Reflexión y auto-mejora

Extiende el loop básico añadiendo un paso de reflexión: después de completar la tarea, el agente evalúa su propio output y lo mejora si es necesario. La reflexión puede ser: critique (identificar problemas en el output actual), revision (corregir los problemas identificados), y verification (verificar que la corrección es correcta). Reflexion (Shinn et al., 2023) demostró que los agentes que reflexionan sobre sus errores y almacenan las lecciones aprendidas en memoria cometen menos errores en tareas similares futuras.

Patrón 4: Agente con memoria persistente

Extiende cualquier patrón base añadiendo memoria a largo plazo: el agente puede recuperar y almacenar información relevante entre sesiones. La memoria persistente puede ser: hechos sobre el usuario/contexto (que el agente aprende a lo largo del tiempo), episodios pasados (lo que funcionó y lo que no en tareas similares), procedimientos (cómo hacer cosas que el agente ha aprendido a hacer). Las implementaciones incluyen mem0 (open source, con API simple), Zep (gestión de memoria long-term para agentes), y vectorstore con TTL para búsqueda semántica de memorias pasadas.

SECCIÓN 3 · CÓMO FUNCIONA

El diseño del estado del agente

El estado del agente es la representación en memoria de todo lo que el agente sabe sobre el progreso actual de la tarea. En LangGraph, el estado es un TypedDict Python que se pasa entre los nodos del grafo y que cada nodo puede modificar. Un estado típico incluye: el objetivo original, el historial de mensajes (conversación con el LLM), el historial de acciones y observaciones, los outputs intermedios de las herramientas, y metadatos del proceso (número de iteraciones, tiempo transcurrido).

La gestión del estado es uno de los desafíos más importantes en sistemas agénticos complejos. El estado puede crecer indefinidamente con cada iteración, eventualmente saturando la ventana de contexto del LLM. Las estrategias de compresión del estado son: resumen periódico del historial (condensar las últimas N iteraciones en un resumen), ventana deslizante (mantener solo las últimas K iteraciones), y extracción de hechos clave (extraer solo la información relevante del historial y descartar el resto).

Human-in-the-Loop (HITL)

El patrón Human-in-the-Loop inserta puntos de verificación donde el agente pausa y solicita aprobación humana antes de continuar. Los tipos de HITL son: checkpoints de aprobación (el agente presenta su plan y espera aprobación antes de ejecutar), interrupciones por baja confianza (el agente pausa cuando su confianza en una decisión está por debajo de un umbral), y supervisión de acciones de alto impacto (el agente siempre requiere aprobación para acciones irreversibles). LangGraph implementa HITL con checkpoints de estado persistente que permiten "pausar" el grafo y reanudar después de la intervención humana.

SECCIÓN 4 · EJEMPLOS REALES

- Plan-and-Execute para análisis de mercado: el agente planificador genera un plan de 7 pasos para analizar el mercado de energía solar en España. El executor ejecuta cada paso (búsqueda web, extracción de datos, análisis financiero). Si el paso 3 falla, el planificador puede replanificar los pasos restantes. El plan visible hace el proceso auditable y facilita la supervisión humana.
- Reflexión en generación de código: un agente genera código Python para una función específica. El reflexor identifica que el código no maneja correctamente el caso donde el input es None. El revisor corrige el bug. El verificador ejecuta los unit tests para confirmar

que la corrección es correcta. El resultado final es código más robusto que el generado en la primera pasada.

- HITL en agente de automatización de RRHH: el agente procesa solicitudes de vacaciones automáticamente para peticiones estándar. Para solicitudes que exceden el saldo disponible o coinciden con periodos de alta actividad, el agente presenta la solicitud a un manager de RRHH para aprobación manual. La decisión de cuándo escalar está codificada en las reglas del agente.
- Memoria persistente en asistente de ventas: el agente almacena preferencias del cliente (formato preferido de propuestas, sensibilidades de precio, decisores clave) descubiertas en conversaciones anteriores. En la siguiente interacción, el agente recupera esta memoria y personaliza su respuesta sin necesitar que el usuario repita el contexto.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Diseñar el estado del agente demasiado grande desde el principio. Un estado con 20+ campos es difícil de mantener y depurar. Empieza con el estado mínimo necesario (objetivo + historial de mensajes) y añade campos solo cuando la lógica del agente los necesite.

Error 2: No implementar compresión del historial en agentes de larga duración. Un agente que procesa un proyecto de 3 horas puede acumular decenas de miles de tokens de historial, eventualmente llenando la ventana de contexto y produciendo comportamiento degradado. Implementa compresión periódica del historial desde el diseño inicial.

Error 3: Ignorar los estados de error en el grafo del agente. Un agente bien diseñado tiene estados explícitos para los casos de fallo: "herramienta falló", "tarea no completable", "confirmación humana requerida". Sin estos estados, los fallos se manifiestan como comportamientos indefinidos que son difíciles de diagnosticar.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

La plantilla de diseño arquitectónico para un agente LLM: define el estado (TypedDict con los campos mínimos necesarios). Define las herramientas (funciones Python con schemas JSON). Implementa el nodo de razonamiento (llamada al LLM con las herramientas). Implementa los nodos de herramientas (funciones que ejecutan las acciones). Define las transiciones (si el modelo pidió una herramienta → ejecutar herramienta; si no → terminar o escalar). Añade los nodos HITL para acciones de alto impacto. Define el estado de error y el comportamiento de fallback.

Para elegir el patrón arquitectónico correcto: tarea simple y flujo conocido → pipeline predefinido. Tarea compleja pero con pasos claros → plan-and-execute. Tarea exploratoria o con resultados intermedios impredecibles → ReAct loop. Tarea de larga duración con necesidad de aprender de errores → ReAct + reflexión. Tarea con usuario que debe mantenerse informado → HITL. Sistema que debe mejorar con el tiempo → memoria persistente.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M10-T3 (Planificación): el plan-and-execute es una aplicación directa de las técnicas de planificación.
- M10-T4 (Memoria): los patrones de memoria persistente y compresión del historial son componentes clave de la arquitectura.
- M10-T7 (Frameworks LangGraph/CrewAI): los frameworks implementan estos patrones arquitectónicos con abstracciones de alto nivel.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los patrones arquitectónicos principales son: ReAct loop (simple, reactivo, para tareas exploratorias), plan-and-execute (plan visible, más auditable, para tareas con pasos claros), reflexión (auto-mejora del output), y memoria persistente (aprender entre sesiones). El estado del agente es el TypedDict que fluye entre nodos; comenzar con estado mínimo y expandir solo cuando es necesario. La compresión del historial es imprescindible para agentes de larga duración. HITL para acciones de alto impacto. LangGraph es el framework de referencia de producción con checkpointing, time-travel debugging y HITL nativo.